

PROJECT REPORT

CS202 - PPP

Title	C language compiler optimisation	
Course	CS202 - PPP	
Name and Entry No.	Shubham Shobhit Gupta Abhijit Kamble Tanish Kumar Moinak Goswami	2024CSB1157 2024CSB1155 2024CSB1401 2024CSB1161 2024CSB1225
Date	12th of May 2026	

1. Project Overview

This project implements a source-level optimizer for C programs using pycparser to parse input C code into an Abstract Syntax Tree (AST), then applies a series of classical compiler optimizations before generating a Control Flow Graph (CFG) for visualization. The pipeline transforms code through multiple passes — Constant Propagation, Constant Folding, Dead Code Elimination, and Unused Assignment Elimination — iterating until the program reaches a fixed point.

Note on CFG coverage: Since the CFG is constructed directly from pycparser's AST rather than from a lowered IR, it may not capture every nuance of the full C language. Constructs such as goto, setjmp/longjmp, function pointers, switch statements with fallthrough, and complex nested expressions may not be fully represented in the control flow graph.

2. Code Preprocessing

Before parsing, the raw C source is cleaned to make it compatible with pycparser, which does not run a real C preprocessor. The preprocessing stage performs the following steps:

- Single-line and multi-line comments are stripped using regex.
- Simple #define macros (value substitution only) are extracted, stored, and replaced inline via word-boundary regex matching.
- #include directives are removed entirely, as pycparser cannot handle standard library headers without stubs.

This approach handles the most common preprocessor patterns found in typical student or algorithmic C programs, though complex macro expansions (e.g. function-like macros, variadic macros) are not supported.

3. Optimization Passes

Four optimization passes are implemented, each operating directly on the pycparser AST nodes:

3.1 Constant Folding

Constant Folding walks the AST and evaluates binary expressions whose both operands are integer constants at compile time. For example, the expression 3 + 5 is replaced with 8. Supported operators include arithmetic (+, -, *, /, %), comparisons (>, <, >=, <=, ==). Division by zero is safely skipped. This pass is applied both before and after each pipeline iteration to clean up any newly created constants.

3.2 Constant Propagation

Constant Propagation tracks variables whose values are known constants and substitutes their values at usage sites. An environment (dictionary) maps variable names to their known integer values. The pass handles:

- Variable declarations with constant initialisers.
- Assignments where the RHS reduces to a constant.
- Conservative merge at if/else branches — a variable is kept constant only if both branches agree on the same value.
- Conservative invalidation after while loops — variables modified in a loop body are marked unknown, since the loop may execute zero or many times.

3.3 Dead Code Elimination (DCE)

The DCE pass removes code that can provably never execute. It works at the statement-block level and handles three main scenarios:

- Constant-true conditions: if (1) { ... } — the else branch is discarded and the then-branch is inlined.
- Constant-false conditions: if (0) { ... } — the entire if body is removed, keeping only the else branch if present.
- Constant-false while loops: while (0) { ... } — the loop is removed entirely.
- Unreachable code after return: any statements following a return or exit()/abort() call are dropped from the block.

3.4 Unused Assignment Elimination (UAE)

UAE performs a backward liveness analysis on each basic block. Starting from the end of a block, it tracks which variables are "live" (will be read before being overwritten). An assignment to a variable that is never subsequently read is eliminated, provided the right-hand side has no side effects (no function calls, no increment/decrement operators). This pass handles assignments, declarations, return statements, and loops conservatively.

---

4. Iterative Optimization Pipeline

Individual optimization passes interact: Constant Propagation may expose new constant expressions for Constant Folding; folding may reveal always-true or always-false conditions for DCE; DCE may leave behind unused variables for UAE. A single pass is therefore insufficient.

The pipeline runs the three core passes in a fixed order (Propagation → Folding → DCE) and repeats until convergence:

$$\text{Entry} \rightarrow (\ \text{Constant Propagation} \rightarrow \text{Constant Folding} \rightarrow \text{DCE}\ )^{n} \rightarrow \text{UAE} \rightarrow \text{Final Folding}$$

At each iteration, the AST is serialized to a string and compared with the previous iteration's string. If the two strings are identical, the AST has reached a fixed point and iteration stops. A safety cap of 50 iterations prevents infinite loops in pathological cases. UAE is applied once after the iterative loop, followed by one final Constant Folding pass to clean up any residual expressions.

Pass	What It Achieves	Run When
Constant Propagation	Replaces known-constant variables with their values	Each iteration
Constant Folding	Evaluates constant expressions at compile time	Each iteration + final
Dead Code Elimination	Removes unreachable and always-false branches/loops	Each iteration
Unused Assignment Elim.	Drops assignments whose results are never read	Once, post-loop

---

5. Output: CFG Images and Pipeline Graph

The project produces a rich set of PNG images to visualise the program at each stage of optimization:

5.1 AST Image

Before any optimization, the AST is converted to a NetworkX directed graph using a recursive traversal and exported as ast.png via Graphviz (pydot). Each node is labelled with the AST node class name (e.g. BinaryOp, If, Return).

5.2 Per-Stage CFG Images

After every individual pass in every iteration, a new CFG PNG is generated with a descriptive filename (e.g. cfg_iter_2_fold.png, cfg_iter_1_dce.png). The CFG builder constructs nodes for each statement with human-readable labels (e.g. "x = a + b;", "if (x > 0)"), adds True/False edge labels on conditionals, back-edges on loops, and merge nodes at if-join points. A global ENTRY node connects to main(). Inter-function call edges are drawn with a "call" label where the callee is a function defined in the same file.

5.3 Pipeline Graph

After all iterations complete, a summary pipeline_graph.png is rendered using Graphviz's Digraph with a left-to-right layout. Each stage (one node per CFG snapshot) is connected in order with an edge labelled by the pass that produced it (e.g. "1_prop", "1_fold", "1_dce", "final_UAE"). This gives a single at-a-glance view of the complete optimization journey.

5.4 Final CFG

A final cfg_optimized.png is generated after all passes are complete, representing the fully optimized program's control flow.

---

6. Known Limitations

- CFG coverage: The CFG is derived from pycparser's AST and does not model goto, switch/case fallthrough, setjmp/longjmp, or indirect function calls via pointers.
- Propagation scope: Constant Propagation is intraprocedural only — values do not propagate across function call boundaries.
- Preprocessing: Function-like macros, conditional compilation (#ifdef), and standard library headers are not supported.
- Loop analysis: Propagation and UAE treat loops conservatively (may miss optimizations inside stable loops).
- Type system: Only integer constants are folded/propagated; floats, pointers, and structs are left untouched.